FORM-TL-01 · Vecindario
Notas de presentación: bloque de publicación y muro

APUNTE PAZMIÑO WASHINGTON GERMAN

Periodo académico 2026–2027

1. Publicación y distribución al muro

Recorrido de un aviso. El usuario o el negocio local crea el aviso desde el cliente (CLI). La primera llamada (S1) envía a PUB el `id_usuario`, el texto, la categoría, la ubicación y el tipo de aviso (normal o alerta); PUB lo registra y responde con el `id_aviso`. Si el aviso lleva imagen, el cliente la sube directamente al servicio externo de almacenamiento (S2), que devuelve la URL o el identificador del archivo, y después registra esa referencia en PUB (S2b). Así el archivo no pasa por los servicios internos.

Cuando el aviso está completo, PUB publica el evento `AvisoPublicado` en Kafka (A1) y no espera nada más: el usuario ya recibió su confirmación. MUR consume el evento (A2), consulta a SEG los seguidores del autor mediante gRPC (S3) y agrega el aviso al muro de cada seguidor. En paralelo, NOT consume el evento `NuevaPublicación` o `AlertaSeguridad` (A3) y envía la notificación push a los seguidores (A5). Cuando un seguidor abre la aplicación, obtiene de MUR su muro ya armado (S10).

ID	Origen → destino	Tipo	Sincronía	Protocolo y middleware
S1	CLI → PUB	Petición-respuesta	Síncrona	HTTPS/TCP, REST API
S2	CLI → almac. externo	Petición-respuesta	Síncrona	HTTPS/TCP, API externa
S2b	CLI → PUB	Petición-respuesta	Síncrona	HTTPS/TCP, REST API
A1	PUB → Kafka	Mensajería	Asíncrona	TCP, Kafka
A2	Kafka → MUR	Mensajería	Asíncrona	TCP, Kafka
S3	MUR → SEG	RPC	Síncrona	HTTP/2, gRPC
A3	Kafka → NOT	Mensajería	Asíncrona	TCP, Kafka
A5	NOT → CLI	Mensajería push	Asíncrona	HTTPS/TCP, FCM
S10	CLI → MUR	Petición-respuesta	Síncrona	HTTPS/TCP, REST API

Mecanismos y justificación.

- **REST sobre HTTPS, síncrono (S1, S2, S2b).** Crear el aviso, subir la imagen y vincularla son acciones en las que el usuario necesita una respuesta inmediata (`id_aviso` o URL) para continuar. HTTPS funciona de forma nativa en Android y en el navegador y protege los datos en tránsito.
- **Kafka, asíncrono (A1, A2, A3).** La distribución al muro y las notificaciones pueden completarse después de responder al usuario. Kafka desacopla a PUB de MUR y de NOT: PUB publica el evento y continúa, y el evento queda guardado aunque un consumidor esté caído.

- **gRPC sobre HTTP/2, síncrono (S3).** MUR necesita cada página de seguidores para seguir distribuyendo. gRPC trabaja con mensajes binarios compactos y un contrato `.proto` entre servicios internos, por lo que tiene menos latencia que REST con JSON.

R3: tiempos distintos para avisos normales y alertas. Un aviso normal puede tardar hasta 5 s en aparecer en el muro de los seguidores, así que se acepta consistencia eventual. Una alerta de seguridad debe aparecer en menos de 1 s. Para que una alerta no quede en cola detrás de los avisos normales, PUB la publica en un tópico propio (`alertas.seguridad`) con consumidores dedicados en MUR y en NOT. La alerta tiene prioridad y capacidad de procesamiento separada del resto.

R4: un negocio con 30.000 seguidores. PUB no espera a que el aviso llegue a los 30.000 seguidores, porque la distribución es asíncrona (A1). MUR pide los seguidores a SEG por páginas (S3) y reparte el trabajo en lotes entre varios consumidores en paralelo. Además, se limita la carga que puede generar un solo autor, para que una publicación masiva no ocupe toda la capacidad y el servicio siga respondiendo al resto de usuarios.

F2: el servicio de muro se cae mientras un negocio publica. La distribución no se pierde. MUR confirma un evento en Kafka (*commit* del *offset*) solo después de procesarlo; si se cae a mitad del trabajo, los eventos no confirmados siguen en Kafka y, al recuperarse, MUR los vuelve a leer desde el último punto confirmado. Como un evento puede procesarse dos veces, el consumidor es idempotente: agregar el mismo aviso al mismo muro no crea un duplicado, por ejemplo gracias a una clave única (`id_usuario`, `id_aviso`).

2. Guion de exposición

Tramo de Washington: de 0:40 a 1:50 (unos 70 segundos).

1. **Publicar con imagen (S1, S2, S2b), unos 15 s.** El cliente crea el aviso en PUB por REST y recibe su identificador. La imagen va directo al almacenamiento externo, que devuelve la URL, y S2b la liga al aviso. Son llamadas síncronas porque el usuario necesita la confirmación.

2. **Kafka (A1), unos 10 s.** Desde aquí todo es asíncrono: PUB publica `AvisoPublicado` en Kafka y no espera, así que publicar no se vuelve lento aunque el autor tenga 30.000 seguidores.

3. **Muro y gRPC (A2, S3), unos 15 s.** MUR consume el evento y pide a SEG los seguidores por páginas con gRPC, una llamada directa y rápida entre servicios. Con cada lote actualiza los muros, con varios consumidores en paralelo.

4. **Alertas, R3, unos 10 s.** Las alertas de seguridad van por un tópico propio con consumidores dedicados: llegan en menos de un segundo, mientras que un aviso normal puede tardar hasta cinco.

5. **Notificaciones (A3), unos 5 s.** NOT escucha el mismo evento y notifica a los seguidores.

6. **Fallo F2, unos 15 s.** Si MUR se cae a mitad de la distribución, no se pierde nada: el evento sigue en Kafka sin confirmar y MUR lo retoma al volver. Como es idempotente, nadie ve el aviso repetido. *Paso la palabra a César.*